Module 6 Tangent: Macros

DSLs (& Macros)

How to design the best or worse system ever created.
There is no in between.

1.

Domain-Specific Languages

“Always select the right tool for the right job.”

Domain-Specific Languages

Definition

A DSL is a programming language built for a specific problem/domain.

- Intentionally more restrictive than many general-purpose languages.
- Well-known DSLs:
  - Regex: String pattern matching.
  - SQL: Database queries/updates.
  - HTML: Web page structure.

Representations

DSLs can be represented in different ways to allow interactions with other languages.

- Strings: DSL code is stored in a string which can be interpreted at runtime (Regex, SQL).
  - Unsafe, messy with quotes/interpolation.
- Libraries: DSL code is *approximated* using existing features of a language (ORMs).
  - Not an accurate DSL; often has quirks.
- Embedding: DSL code is written directly; integrating with the “host” language.
  - Hard to do, harder to do well.

```java
var regex = "[a-z]+";
"cat".matches(regex); //true
"kit-kat".matches(regex); //false
```

```java
var usr = "username";
var pwd = "password";
var sql =
  "SELECT * FROM users\n" +
  "WHERE username = '"+usr+"'\n" +
  "AND password = '"+pwd+"'\n";
db.execute(sql); //ResultSet...
```

```jsx
function Hello({name}) {
  return (
    <h1>Hello, {name}!</h1>
  );
}
```

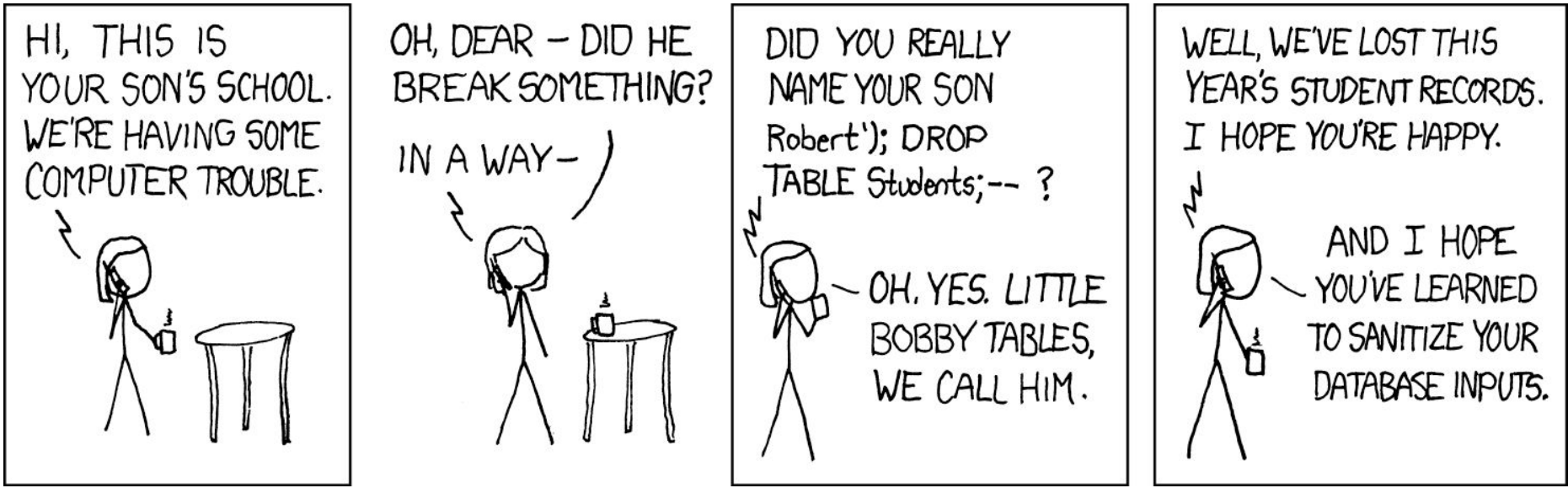
```jsx
function Hello({name}) {
  return React.createElement(
    "h1",
    {},
    "Hello, ",
    name,
    "!",
  );
}
```

Domain-Specific Languages

2.

Code Injection

Robert'); DROP TABLE Students;--



SQL Injection

Code Injection

Definition

A category of security vulnerabilities where user input is evaluated as code.

1. The user submits malicious input.
2. The application inserts this input into text representing code (!).
3. The malicious input is evaluated as malicious code.

Solution

User input needs to be *sanitized* before being inserted into a code context.

- Special characters (especially quotes) need to be escaped.
- Better yet, let a library handle this and use a safe API that takes structured arguments.
  - e.g. SQL PreparedStatements

```java
var usr = "username";
var pwd = "' OR 1=1 --";
var sql =
  "SELECT * FROM users\n" +
  "WHERE username = '"+usr+"'\n" +
  "AND password = '"+pwd+"'\n";
```

```java
var usr = "username";
var pwd = "' OR 1=1 --";
var stmt = conn.prepareStatement(
  "SELECT * FROM users\n" +
  "WHERE username = ?\n" +
  "AND password = ?\n"
);
stmt.setString(1, usr);
stmt.setString(2, pwd);
```

```python
var sql = """
  SELECT * FROM users
  WHERE username = 'username'
  AND password = '' OR 1=1 --'
  """;
```

```python
var sql = """
  SELECT * FROM users
  WHERE username = 'username'
  AND password = '\' OR 1=1 --'
  """;
```

SQL Injection

3.

Implementing DSLs

Code is Data, Data is Code

Implementing DSLs

String DSLs

The “easy” approach is to use a String for DSL code that is evaluated at runtime.

- e.g. Regex, SQL
- Great if there’s an existing runtime!
- Validation of syntax/semantics is effectively limited to runtime.
  - IDEs might have limited support.
- Vulnerable to code injection when working with argument values.

DSL Libraries

Create a new library that emulates the syntax/semantics of the original DSL.

- Abstract the DSL behind a normal API.
  - Not a DSL, just a library performing the equivalent (ish) function of the DSL.
  - Library will introduce quirks/differences.
  - Still tends to be limited on validation.
- Ab(use) language features to customize syntax for maximum DSL-ness.
  - All of the problems of the above, plus even more from wonky syntax.

```kotlin
//SQL, Kotlin

val name = "Ash"
val pokemon = DB.query("""
    SELECT *
    FROM trainers t, pokemon p
    WHERE t.name = ${trainer}
    AND t.id = p.trainerId;
""");

//Unstructured data :(
//[SqlRow "Pikachu" …, …]
```

```haskell
//Esqueletto, Haskell

let name = "Ash"
pokemon ← select $ do
  (trainer, pokemon) ← from $
    table @Trainers
      `innerJoin` table @Pokemon
        `on` (\(t :& p) →
          t ^. #id ==. p ^. #trainerId
  where_ (trainer ^. Name ==. val name)
  pure $ pokemon

-- Structured (typed) data!
-- [Entity Pokemon "Pikachu" …, …]
```

Tagged Templates

Idea

Let's "split" the String into a template that separates literals from values.

- Consider `` `x = ${x}` ``:
  - Literal `"x = "`
  - Value x
  - Literal `""` (n values, n + 1 literals)
- Our template can thus handle values separately from literal source code.
  - And supports non-string values!

Macros

We can "solve" the compile-time issue by leveraging macros!

- Macros run at compile-time, so we can process (& validate) the DSL function then.
  - Compile-time syntax/semantics checks.
  - Less overhead at runtime too!
- Primary limitation is editor support; proper syntax highlighting & IntelliSense is useful.
  - Difficult, but feasible - requires an understanding of embedded languages.

Module 6 Tangent: Macros

DSLs (& Macros)

How to design the best or worse system ever created.
There is no in between.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals